

WEEK 02 · LAB ACTIVITY GUIDE · 60 MINUTES

From Problem to Running Program

Check your GitHub set-up, build a program by hand and in Visual Studio, study a fully worked problem, then solve a new one and push it to your Week 02 repository

Name / Matric No.	
Section / Date	
GitHub username	
You need	Lab PC with Visual Studio 2022 (Desktop development with C++), Draw.io (app.diagrams.net), your GitHub account
Before this	Git pre-reading and slides (first 30 minutes of the lab). The lecture may come before or after this lab: the Concept boxes and the worked example give you enough theory to start.

Lab learning outcomes

By the end of this lab you should be able to:

#	Outcome	CLO
LO1	Set up a GitHub repository for the week and verify that the local and remote repositories are connected	CLO2
LO2	Use a command-line C++ compiler to compile, link, load and execute a program, and explain what each stage produces	CLO1
LO3	Relate each command-line stage to the matching action in Microsoft Visual Studio	CLO2
LO4	Identify the input, process and output (IPO) of a problem and draw its structure chart	CLO3
LO5	Design an algorithm with pseudocode and a Draw.io flowchart, and desk-check it with test data	CLO3
LO6	Code and test the algorithm, classifying errors as syntax, link, run-time or logic errors, and push the work to GitHub	CLO1

Time plan

Part	Activity	Minutes
—	Recall	2
A	GitHub readiness: account, Week 02 repository, local ↔ remote connection	8
B	Compile, link, load and execute with the command line (<code>cl.exe</code>)	11
C	The same pipeline inside Visual Studio	7
D	Worked example: Download Time Calculator (study, type, run)	10
E	Your turn: Streaming Data Usage (analysis → design → code → test)	17
F	Commit, push and verify on GitHub	5
	Total	60

RECALL · answer what you can now (2 min)

Write short answers. You will answer the same five questions again at the end of the lab (and your lecturer closes the Week 02 lecture with them).

1. What are the two phases and four steps of the programming process?
2. Give the input, the process and the output of “convert a distance in miles to kilometres”.
3. Which flowchart symbol is used for input/output, and which for a decision?
4. The computer cannot run a `.cpp` file directly. Which tools turn it into a running program?
5. Which Git command sends your commits to GitHub?

Part A — GitHub readiness check (8 min)

Everything you produce today is submitted through GitHub, so make sure the connection works **before** you start. Do not move to Part B until every row of the checklist at the end of this part is ticked.

A1. Confirm your GitHub account

1. Go to **github.com** and sign in. Write your username on the cover page.
2. No account yet? Create one now (Git Module, pages 4–5), preferably with your student e-mail, and choose the **Free** plan.
3. In Visual Studio, check you are signed in to GitHub: profile icon (top right) › your GitHub account should be listed. If not: **Add another account › GitHub** (Git Module, page 7).

A2. Create the Week 02 repository on GitHub

1. On github.com click + (top right) › **New repository**.

Setting	Value
Repository name	BITP1113-Week02
Description	BITP 1113 Programming Technique - Week 02 lab
Visibility	As instructed by your lecturer. If Private , add your lecturer as a collaborator: <i>Settings › Collaborators</i>
Add a README file	✓ ticked
Add .gitignore	Template: VisualStudio

- Click **Create repository**, then **Code** › copy the HTTPS URL, e.g.
<https://github.com/<username>/BITP1113-Week02.git>.

A3. Clone it to the lab PC

Use **one** of these. Both create the folder `C:\BITP1113\BITP1113-Week02` linked to GitHub.

Visual Studio	Developer Command Prompt
Start window › Clone a repository › paste the URL › Path: <code>C:\BITP1113\BITP1113-Week02</code> › Clone	<code>mkdir C:\BITP1113</code> <code>cd /d C:\BITP1113</code> <code>git clone <your-URL></code>

A4. Verify the local ↔ remote connection

Open the **Developer Command Prompt for VS 2022** and run each command. Compare with the expected result.

Command	Expected result
<code>cd /d C:\BITP1113\BITP1113-Week02</code>	The prompt now shows your repository folder
<code>git config user.name</code> and <code>git config user.email</code>	Your name and the e-mail of your GitHub account. If blank, set them (pre-reading section 4)
<code>git remote -v</code>	Two lines: <code>origin</code> <code>https://github.com/<username>/BITP1113-Week02.git (fetch) and (push)</code>
<code>git status</code>	On branch <code>main</code> and <code>Your branch is up to date with 'origin/main'</code>

Now prove that a change travels all the way to GitHub. Open `README.md` (e.g. `notepad README.md`), add your name, matric number and section, save, then:

round-trip test
<code>git add README.md</code> <code>git commit -m "Add student details to README"</code> <code>git push</code>

If a browser window asks you to sign in to GitHub on your first push, sign in and authorise it. Then **refresh your repository page on github.com**: your details must appear in the README.

If something goes wrong

Message	Cause and fix
'git' is not recognized	You are in a normal Command Prompt. Use the Developer Command Prompt for VS 2022 , or Visual Studio's Git menu
fatal: not a git repository	You are not inside the cloned folder. <code>cd /d C:\BITP1113\BITP1113-Week02</code>
Author identity unknown	Run <code>git config --global user.name "..."</code> and <code>git config --global user.email "..."</code> , then commit again
Authentication failed / Permission denied	Sign in again when the browser prompt appears, or re-add your GitHub account in Visual Studio. Check the URL has <i>your</i> username
rejected ... (fetch first)	GitHub has commits you don't have (e.g. you edited on the website). Run <code>git pull</code> , then <code>git push</code>

Readiness checklist

Check	✓	Demonstrator initial
I can sign in to my GitHub account		
Repository BITP1113-Week02 exists on GitHub with README and VisualStudio .gitignore		
Repository cloned to C:\BITP1113\BITP1113-Week02		
<code>git remote -v</code> shows my repository URL; <code>git status</code> says up to date with origin/main		
My README change is visible on github.com (round trip works)		

Part B — Build a program by hand with cl.exe (11 min)

KEY IDEA · Concept: what happens when you “build”

A C++ program goes through five stages before you see any output:

- **Preprocess** — the preprocessor handles lines starting with `#`, e.g. it pastes the contents of `<iostream>` into your code.
- **Compile** — the compiler checks the syntax and translates your code into machine code, an **object file** (`.obj`). Syntax errors are reported here.
- **Link** — the linker joins your object file with library code (such as `cout`) into one **executable** (`.exe`). Missing functions are reported here.
- **Load** — when you run the `.exe`, the operating system’s loader copies it into memory (RAM).
- **Execute** — the CPU runs the instructions one by one. Run-time and logic errors appear here.

Visual Studio does all of this when you press one key. Here you do each stage yourself with `cl.exe`, the same Microsoft compiler Visual Studio uses behind the scenes.

B1. Write the source file

In the Developer Command Prompt, make a `PartB` folder inside your repository and create the file:

```
cd /d C:\BITP1113\BITP1113-Week02
mkdir PartB
cd PartB
notepad bandwidth.cpp
```

Choose **Yes** to create the file, type the program below and save it. It converts an internet speed in **megabits** per second into **megabytes** per second (8 bits = 1 byte), a conversion you will need again in Parts D and E.

PartB\bandwidth.cpp

```
// bandwidth.cpp - converts an internet speed from Mbps to MB/s
#include <iostream>
using namespace std;

double toMBps(double mbps);    // function prototype

int main()
{
    double mbps;
    cout << "Enter your Wi-Fi speed (Mbps): ";
    cin >> mbps;
    cout << "That is " << toMBps(mbps) << " MB per second" << endl;
    return 0;
}

double toMBps(double mbps)    // function definition
{
    return mbps / 8.0;        // 8 bits = 1 byte
}
```

B2. Run each stage separately

Stage	Type this	What you should see / check
Preprocess (look only)	<code>cl /EHsc /P bandwidth.cpp</code>	A new file <code>bandwidth.i</code> : your code after <code>#include</code> has been pasted in. Count its lines with <code>find /c /v "" bandwidth.i</code>
Compile	<code>cl /EHsc /c bandwidth.cpp</code>	<code>bandwidth.obj</code> appears. <code>/c</code> means <i>compile only, do not link</i>
Link	<code>link bandwidth.obj</code>	<code>bandwidth.exe</code> appears
Load + execute	<code>bandwidth.exe</code>	Enter <code>100</code> . Output: <code>That is 12.5 MB per second</code>
Check	<code>dir bandwidth.*</code>	You should now have <code>.cpp</code> , <code>.i</code> , <code>.obj</code> and <code>.exe</code> files

`/EHsc` turns on standard C++ exception handling; without it `cl` prints a warning about `<iostream>`. `/P` only preprocesses (no `.obj`), which is why compiling is a separate step. In everyday use the single command `cl /EHsc bandwidth.cpp` compiles and links in one go.

B3. Break it on purpose: which stage complains?

Make each change, rebuild, read the message, then undo the change before trying the next one.

Change to make	Rebuild with	Stage that reports it	Error code / what you saw
Delete the <code>;</code> after <code>cin >> mbps</code>	<code>cl /EHsc /c bandwidth.cpp</code>		
Rename the function definition to <code>toMBps</code> (lower-case b) but keep the prototype and the call as <code>toMBps</code>	<code>cl /EHsc bandwidth.cpp</code>		
Change <code>/ 8.0</code> to <code>* 8.0</code>	<code>cl /EHsc bandwidth.cpp</code> then run		

THINK

The second change compiles without complaint. Why does only the **linker** notice that `toMBps` has no body? (Hint: the prototype promises the compiler that the function exists *somewhere*.)

The third change builds and runs. Which kind of error is it, and which step of the programming process would catch it?

Finally, delete the build output so it is not pushed later: `del bandwidth.i bandwidth.obj bandwidth.exe`

Part C — The same pipeline in Visual Studio (7 min)

1. Visual Studio › **Create a new project** › **Console App (C++)**. Project name **Bandwidth**, Location **C:\BITP1113\BITP1113-Week02**, Solution name **Lab02** › **Create**. Because the folder is a Git repository, Visual Studio connects the solution to it automatically.
2. Replace the contents of **Bandwidth.cpp** with your **bandwidth.cpp** code.
3. Show the commands Visual Studio runs: **Tools** › **Options** › **Projects and Solutions** › **Build and Run** › **MSBuild project build output verbosity = Detailed**.
4. Perform each action and tick it. After each build, search the **Output** window (Ctrl+F) for **CL.exe** and **link.exe**.

Command line (Part B)	Visual Studio action	Shortcut	✓
<code>notepad bandwidth.cpp</code>	Edit the file in the code editor		
<code>cl /EHsc /c bandwidth.cpp</code> (compile only)	Build › Compile	Ctrl+F7	
<code>cl + link</code> (compile and link)	Build › Build Solution	F7	
<code>bandwidth.exe</code> (load + execute)	Debug › Start Without Debugging	Ctrl+F5	
Compiler/linker messages in the console	Error List and Output windows	Ctrl+\\, E	
<code>del *.obj *.exe</code>	Build › Clean Solution		
The <code>/EHsc</code> and other options	Project › Properties › C/C++ › Command Line and Linker › Command Line		

5. Repeat the missing-semicolon error from B3 in Visual Studio. Error code in the Error List: _____ Same as the command line? _____

NOTE · Why this matters

When a build fails, ask “did the **compiler** or the **linker** complain?” Codes starting with **C** (e.g. C2143) come from the compiler; codes starting with **LNK** (e.g. LNK2019) come from the linker. Knowing the stage tells you where to look for the mistake.

Part D — Worked example: Download Time Calculator (10 min)

KEY IDEA · Concept: the programming process

Programming has **two phases** and **four steps**. **Problem solving phase:** Step 1 *Analysis* (what are the input, process and output?) and Step 2 *Design* (write the algorithm as pseudocode and a flowchart, then desk-check it). **Implementation phase:** Step 3 *Coding* (translate the algorithm into C++ and compile it) and Step 4 *Testing* (run it on sample data and fix errors).

Study how each step is done below. In Part E you will apply exactly the same steps to a new problem, so notice the pattern, not just the answer.

DO THIS · Problem statement

Aiman lives in a UTeM hostel and wants to download the recorded Programming Technique lecture before the Wi-Fi gets busy at night. The video is 500 MB and a speed test shows the hostel Wi-Fi at 20 Mbps.

Internet providers quote speeds in **megabits per second (Mbps)**, but files are measured in **megabytes (MB)**, and 1 byte = 8 bits.

Write a program that asks the user for the file size in megabytes and the connection speed in megabits per second, then calculates and displays the estimated download time in seconds and in minutes.

Step 1: Analysis

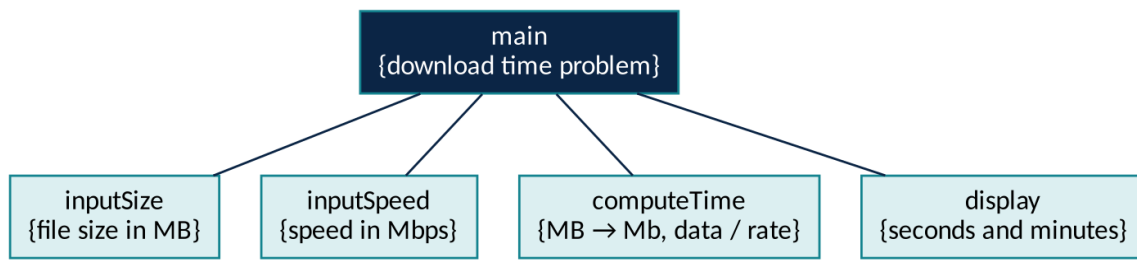
Highlighted: data = *file size (MB), speed (Mbps), download time (seconds, minutes)*; actions = *calculate, display*; hidden requirement = *1 byte = 8 bits*.

	Worked answer
Input	file size in megabytes: <code>sizeMB</code> ; connection speed in megabits per second: <code>speedMbps</code>
Process	convert size to megabits: <code>sizeMb = sizeMB × 8</code> calculate time: <code>seconds = sizeMb / speedMbps</code> convert to minutes: <code>minutes = seconds / 60</code>
Output	download time: <code>seconds</code> and <code>minutes</code>

WATCH OUT · The trap this example avoids

Calculating `500 / 20` gives 25 seconds, eight times too fast, because megabytes and megabits were mixed. Writing units next to every item in the IPO table is what catches this.

Using top-down design, each job becomes a module (function):



Structure chart: main calls four modules.

Step 2: Design

KEY IDEA · Pseudocode style used in this course

We follow the Cal Poly pseudocode standard (J. Dalbey): **keywords in CAPITALS** (**READ**, **DISPLAY**, **COMPUTE ... as ...**, **CALL ... with ... RETURNING ...**, **RETURN**), **one action per line**, the body of each module **indented**, and words from the problem rather than from C++. No step numbers and no Start/Stop: indentation shows the structure, and Start/Stop belong in the flowchart. The supplementary reading, section 4.1, has the full rules.

main is written first as a list of **CALLs**; each module follows as its own block.

pseudocode (top-down)

```

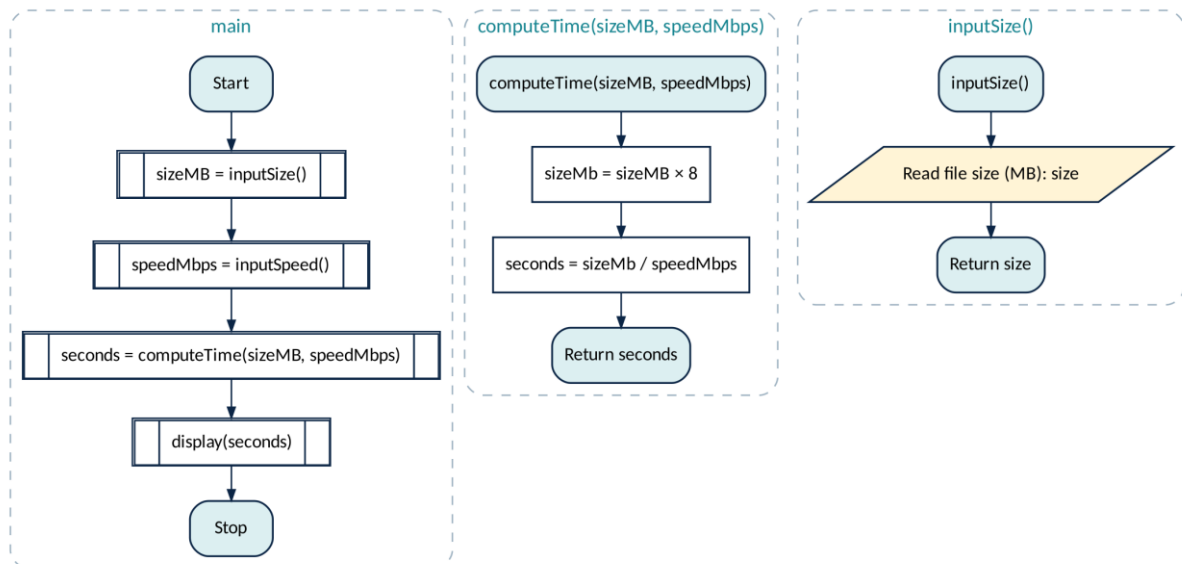
main
  CALL inputSize RETURNING sizeMB
  CALL inputSpeed RETURNING speedMbps
  CALL computeTime with sizeMB and speedMbps RETURNING seconds
  CALL display with seconds

inputSize
  DISPLAY prompt for the file size in megabytes
  READ sizeMB
  RETURN sizeMB

inputSpeed
  DISPLAY prompt for the connection speed in megabits per second
  READ speedMbps
  RETURN speedMbps

computeTime with sizeMB and speedMbps
  COMPUTE sizeMb as sizeMB × 8          (1 byte = 8 bits)
  COMPUTE seconds as sizeMb / speedMbps
  RETURN seconds

display with seconds
  COMPUTE minutes as seconds / 60
  DISPLAY seconds as the download time in seconds
  DISPLAY minutes as the download time in minutes
  
```



Flowcharts: main (predefined-process symbols for each call), computeTime, and one input module. inputSpeed and display follow the same pattern.

Desk-check before coding: work out the expected answers by hand, then trace the pseudocode.

Test	sizeMB	speedMbps	sizeMb	seconds	minutes
1 (Aiman's video)	500	20	4000	200	3.33
2 (Visual Studio installer)	4096	100	32768	327.68	5.46

Step 3: Coding

In Visual Studio, **right-click** Solution 'Lab02' › **Add** › **New Project** › **Console App**, name it **DownloadTime**, then **right-click** **DownloadTime** › **Set as Startup Project**. Type the program, then build with **F7**. Notice that every pseudocode line has been kept as a `//` comment directly above the C++ statement that implements it: a habit the pseudocode standard recommends, and one you will use in Part E.

DownloadTime.cpp (worked example)

```

// download_time.cpp - Lab 02 worked example: Download Time Calculator
// Each // line in a function body is the pseudocode step it implements.
#include <iostream>
using namespace std;

double inputSize();           // reads file size (MB)
double inputSpeed();          // reads speed (Mbps)
double computeTime(double sizeMB, double speedMbps); // returns seconds
void display(double seconds); // shows the result

int main()
{
    double sizeMB, speedMbps, seconds;

    // CALL inputSize RETURNING sizeMB
    sizeMB = inputSize();
    // CALL inputSpeed RETURNING speedMbps
    speedMbps = inputSpeed();
    // CALL computeTime with sizeMB and speedMbps RETURNING seconds
    seconds = computeTime(sizeMB, speedMbps);
    // CALL display with seconds

```

```

        display(seconds);

        return 0;
    }

    double inputSize()
    {
        double sizeMB;
        // DISPLAY prompt for the file size in megabytes
        cout << "Enter file size (MB)          : ";
        // READ sizeMB
        cin >> sizeMB;
        // RETURN sizeMB
        return sizeMB;
    }

    double inputSpeed()
    {
        double speedMbps;
        // DISPLAY prompt for the connection speed in megabits per second
        cout << "Enter connection speed (Mbps): ";
        // READ speedMbps
        cin >> speedMbps;
        // RETURN speedMbps
        return speedMbps;
    }

    double computeTime(double sizeMB, double speedMbps)
    {
        // COMPUTE sizeMb as sizeMB x 8  (1 byte = 8 bits)
        double sizeMb = sizeMB * 8.0;
        // COMPUTE seconds as sizeMb / speedMbps
        double seconds = sizeMb / speedMbps;
        // RETURN seconds
        return seconds;
    }

    void display(double seconds)
    {
        // COMPUTE minutes as seconds / 60
        double minutes = seconds / 60.0;
        // DISPLAY seconds as the download time in seconds
        cout << "Estimated download time      : " << seconds << " seconds" << endl;
        // DISPLAY minutes as the download time in minutes
        cout << "                               = " << minutes << " minutes" << endl;
    }
}

```

Step 4: Testing

Run with **Ctrl+F5** using the desk-check values. Both actual outputs must match the expected ones.

Test	Input	Expected	Actual	Pass?
1	500 MB, 20 Mbps	200 s = 3.33 min	200 seconds = 3.33333 minutes	✓
2	4096 MB, 100 Mbps	327.68 s = 5.46 min	327.68 seconds = 5.46133 minutes	✓
3	500 MB, 0 Mbps	cannot be calculated	inf seconds	run-time issue

THINK · What test 3 teaches

Dividing by a speed of 0 does not crash; it prints `inf`, a meaningless result. Today's programs cannot reject bad input. In Week 4 you will add a **decision** (selection) to check the speed first.

Part E — Your turn: Streaming Data Usage (17 min)

Solve this new problem with the same four steps as Part D. Everything you produce goes into your repository and is pushed in Part F.

DO THIS · Problem statement

Nurul attends her lectures online from home using her phone's mobile data. A 90-minute class streams video at **2.5 Mbps** (megabits per second), and she wants to know how much of her monthly quota each class uses.

Write a program that asks the user for the **streaming bitrate in Mbps** and the **duration in minutes**, then calculates and displays the **data used in megabytes (MB)** and in **gigabytes (GB)**.

Use: 1 minute = 60 seconds, 1 byte = 8 bits, and 1 GB = 1024 MB.

Requirements

Item	Requirement
Design method	Top-down design with four modules : <code>inputBitrate</code> , <code>inputMinutes</code> , <code>computeDataMB</code> , <code>display</code>
Visual Studio project	<code>StreamingData</code> , added to the <code>Lab02</code> solution (as you did for <code>DownloadTime</code>)
Design file	<code>design/StreamingData_design.md</code> in your repository (template below)
Flowchart	<code>design/StreamingData_flowchart.png</code> exported from Draw.io: <code>main</code> plus the <code>computeDataMB</code> module
Tests	Test 1: 2.5 Mbps for 90 minutes. Test 2: values you choose. Test 3: a bitrate of 0

NOTE · Hints: compare with the worked example

- Part D computed **time = data ÷ rate**. Here you need the reverse: **data = rate × time**, so the time must first be in **seconds**.
- The worked example converted MB → Mb by multiplying by 8. Here the result comes out in megabits, so you go the other way: Mb → MB.
- Write the pseudocode in the same style as Part D: capitalised keywords, `main` as a list of **CALLs**, one indented block per module, one action per line.
- Start your code from a copy of `DownloadTime.cpp`. Replace the `//` pseudocode comments with **your** pseudocode lines, then write the C++ under each one. Change the IPO and pseudocode **first**, then the code.

E1–E4. Analysis and design (in the design file)

- In Solution Explorer, right-click the solution › **Open Folder in File Explorer**, create a folder `design`, and in it create `StreamingData_design.md` with Notepad. Copy the template below and complete every section.
- Draw the flowchart in **Draw.io** (Flowchart shape library: Terminator, Data, Process, Predefined Process). **File › Export as › PNG** → `design\StreamingData_flowchart.png`.
- Complete the desk-check in the design file **before** writing any code.

design\StreamingData_design.md (template)

```
# Streaming Data Usage - Design
Name: _____ Matric No: _____ Section: _____

## Step 1: Analysis
Input : ...
Process : ... (write every formula, with units)
Output : ...

Structure chart (module names and jobs):
main -> inputBitrate {...}, inputMinutes {...}, computeDataMB {...}, display {...}

## Step 2: Design
### Pseudocode
main
    CALL inputBitrate RETURNING bitrateMbps
    CALL inputMinutes RETURNING ...
    CALL computeDataMB with ... RETURNING ...
    CALL display with ...

inputBitrate
    ...

inputMinutes
    ...

computeDataMB with bitrateMbps and minutes
    COMPUTE seconds as ...
    ...
    RETURN dataMB

display with dataMB
    ...
### Flowchart
See StreamingData_flowchart.png

### Desk-check


| Test | bitrateMbps | minutes | seconds | dataMb | dataMB | dataGB |
|------|-------------|---------|---------|--------|--------|--------|
| 1    | 2.5         | 90      |         |        |        |        |
| 2    |             |         |         |        |        |        |
| 3    | 0           | 60      |         |        |        |        |



## Step 4: Testing


| Test | Expected (MB / GB) | Actual output | Pass? |
|------|--------------------|---------------|-------|
| 1    |                    |               |       |
| 2    |                    |               |       |
| 3    |                    |               |       |


```

E5. Coding

1. Add project **StreamingData** to the solution, set it as the Startup Project, and write **StreamingData.cpp** with the four required functions. Put each pseudocode line in as a **//** comment and the matching C++ statement under it, as in **DownloadTime.cpp**.
2. Build with **F7** until there are no errors. If you meet a C-code or LNK-code error, note which stage reported it.

E6. Testing

1. Run the three tests with **Ctrl+F5** and fill in the Testing table in your design file.

- If an actual result differs from the desk-check, decide whether the error is in the **algorithm** (fix the design first) or in the **code**.

DO THIS · Extension (if you finish early)

Add a third input, the number of classes per week, and display the data used per week in GB. Update the IPO, pseudocode and flowchart before the code.

Part F — Commit, push and verify (5 min)

Use **either** Visual Studio **or** the command line, from the repository folder.

Command line	Visual Studio 2022
<code>git status</code>	View › Git Changes : check the list of changed files
<code>git add .</code>	(staging happens automatically with <i>Commit All</i>)
<code>git commit -m "Lab02: build pipeline, worked example, streaming data"</code>	Git › Commit or Stash , type the message, Commit All
<code>git push</code>	Git › Push

WATCH OUT · Before you commit

The change list must **not** contain `.exe`, `.obj`, `.i` files or `x64/`, `Debug/` or `.vs/` folders. If any appear, delete those build files (or **Build › Clean Solution**) and check that your `.gitignore` is the VisualStudio template.

Open your repository on **github.com** and confirm that it now contains:

```
BITP1113-Week02/
|-- README.md                your details (Part A)
|-- .gitignore               VisualStudio template
|-- PartB/bandwidth.cpp      built by hand (Part B)
|-- Lab02.sln
|-- Bandwidth/               Bandwidth.cpp, Bandwidth.vcxproj ... (Part C)
|-- DownloadTime/           DownloadTime.cpp ... (Part D)
|-- StreamingData/           StreamingData.cpp ... (Part E)
`-- design/
    |-- StreamingData_design.md IPO, pseudocode, desk-check, tests
    `-- StreamingData_flowchart.png
```

Click **Commits** on the repository page: you should see at least two commits (README test and Lab02 work). Show this page to your demonstrator before you leave.

Concept ↔ Practice Map

The same table closes the Week 02 lecture. Whichever session you attended first, this shows how the two halves fit.

Lecture concept (Week 02)	Worked in this lab	You produced	Part
Step 1 Analysis: IPO, structure chart, top-down design	Download time IPO and structure chart	Streaming data IPO and modules in the design file	D, E1
Step 2 Design: algorithm, pseudocode, flowchart symbols	Pseudocode and flowcharts	Pseudocode; Draw.io flowchart PNG	D, E1–E2
Checking the algorithm for logic errors	Desk-check table	Your desk-check	D, E3
Step 3 Coding: compile and correct syntax errors	<code>bandwidth.exe</code> by hand; <code>DownloadTime</code> in VS	<code>StreamingData.cpp</code>	B, C, E5
Step 4 Testing: run-time and logic errors	Errors classified by stage; test table	Your test table	B3, D, E6
Version control for your programs (pre-reading)	GitHub readiness check	Week 02 repository pushed	A, F

Exit checkpoint: answer the recall questions again

Q	Your answer now
1	
2	
3	
4	
5	

Before you leave

- ☐ Readiness checklist (Part A) initialled by the demonstrator
- ☐ `bandwidth.exe` built with `cl` and `link`; three B3 errors recorded
- ☐ Visual Studio actions in Part C ticked
- ☐ `DownloadTime` builds and passes tests 1–2
- ☐ `StreamingData` design file and flowchart complete; program passes your tests
- ☐ Everything pushed; commits visible on github.com